docker-dev — Specifica funzionale

Versione: 0.1
Data: 13 ottobre 2026
Stato: bozza consolidata della baseline; non implementazione collaudata.

Le decisioni approvate derivano dalla discussione del progetto. Le raccomandazioni non approvate restano marcate APERTO. La CR originale e la revisione 2 sono documenti storici, non più riferimenti normativi. Il prototipo è una baseline comportamentale, non uno schema da riprodurre senza modifiche.

1. Obiettivi

Eseguire progetti di sviluppo containerizzati mediante Compose come unico backend di esecuzione, con configurazione di linguaggio riutilizzabile, aggiunte di progetto e servizi opzionali. Preparazione di immagini e volumi può continuare a usare comandi Docker diretti.

La prima versione non deve implementare un orchestratore generale, un interprete di espressioni, un traduttore arbitrario docker run → Compose o la verifica dell'integrità del filesystem interno.

2. Terminologia

- Progetto: directory di lavoro e identità logica da definire.
- Linguaggio: configurazione in langs/<tipo>/.
- Servizio: dipendenza implementata nel catalogo services/<tipo>/.
- Runtime: proprietà di creazione del container dev.
- Container persistente: non rimosso automaticamente all'uscita; non implica processo sempre attivo.
- Container effimero: rimosso all'uscita.
- Compatibilità di creazione: corrispondenza fra configurazione richiesta e configurazione registrata alla creazione, non integrità o salute applicativa.

3. Requisiti

ID	Requisito	Decisione	Test
FR-001	Usare Compose per eseguire dev e servizi	A01	IT-001
FR-002	Comporre runtime di linguaggio e progetto	A02	UT-001
FR-003	Preservare runtime Python /app, venv, uv-python, entrypoint e bashrc	A03	IT-002
FR-004	Nessuna pubblicazione implicita di porte	A04	IT-003
FR-005	Risolvere servizi e alias da dati, senza logica specifica nel resolver	A05	UT-004

ID	Requisito	Decisione	Test
FR-006	Supportare container effimeri e persistenti	A06	IT-004
FR-007	Consentire riavvio di container persistenti fermi e compatibili	A07	IT-005
FR-008	Non eliminare automaticamente container omonimi	A07	IT-006
FR-009	Verificare identità e compatibilità prima del riutilizzo	A08	IT-005
FR-010	Preservare argv del comando senza eval	A09	UT-005
FR-011	Non duplicare dev con up generale seguito da run	A01	IT-001
FR-012	Fornire config, piano/dry-run, arresto e ricreazione esplicita	O03/O09	IT-007
FR-013	Conservare stato sufficiente per arrestare lo stack senza ripetere servizi	O09	IT-008
FR-014	Gestire errori di configurazione senza sovrascritture silenziose	A10	UT-002
FR-015	Delegare preparazione specifica agli script/hook pertinenti	A11	IT-009

4. Operazioni e comportamento

L'interfaccia concreta è aperta (O03); sono richieste le seguenti capacità:

1. Sessione senza servizi: preparare ambiente e creare/riutilizzare dev.
2. Sessione con servizi: prepararli, garantire readiness secondo O07, poi eseguire dev.
3. Container attivo compatibile: attach o exec espliciti secondo operazione; nessuna seconda sessione condivisa automatica.
4. Container incompatibile: mostrare motivazione e richiedere ricreazione esplicita.
5. Container estraneo/identità ignota: errore, oppure adozione esplicita da specificare.
6. Comando diverso da quello originario: non trattare start come sostituzione del comando; eseguire task distinto o exec secondo O05.
7. Config/dry-run: nessun build, volume creato o container avviato.
8. Cleanup: selettivo, confermato, distinto dalla normale chiusura della sessione.

5. Compatibilità Python

Preservare immagine risolta, nome/hostname secondo politica identità, utente dell'immagine, /app, /app/.venv, uv-python nella home dell'utente e shell dell'entrypoint senza comando. La porta 8000 nel template precedente è un refuso da eliminare, non un requisito.

La rete Compose è una differenza intenzionale rispetto al prototipo. Testare DNS, servizi e accesso host; non promettere identità di tutti i metadati Docker.

La sincronizzazione uv resta quella del prototipo, salvo decisione separata: virtualenv non vuoto può saltare sync; UV_SYNC_ALWAYS va preservato.

6. Requisiti non funzionali

- Determinismo del modello e del fingerprint.
- Configurazione leggibile, schema limitato e versionato.
- Errori diagnostici e nessuna esecuzione di stringhe configurabili come codice.
- Segreti non mostrati in diff o log; persistenza protetta secondo O10.
- Concorrenza sul progetto gestita secondo O11.
- Dipendenze e feature Compose controllate prima delle operazioni interessate.

7. Accettazione

Ogni requisito deve avere test verificabile. Le scelte aperte bloccanti devono essere approvate prima della milestone pertinente. La riuscita dei test statici non equivale al collaudo runtime.